```
% ======================================================================
% Dodatek B – Dokumentacja techniczna (WZÓR, ok. 2 strony)
% Wstaw po dodatku A. Wymaga pakietów: booktabs, tabularx, listings (listingi B.1, B.2).
% Plan i zwroty: materialy/07_plan_dodatki_streszczenie_skroty.md.
% ======================================================================

\chapter{Dokumentacja techniczna}
\label{app:dokumentacja}

Dokumentacja jest przeznaczona dla osoby instalującej, utrzymującej lub rozwijającej
system. Kod źródłowy znajduje się w folderze \texttt{projekt}, a szczegółowe opisy -- w
plikach \texttt{README.md} folderów \texttt{projekt} i
\texttt{projekt/modele\_segmentacja}.

\section{Środowiska i instalacja}

System korzysta z dwóch środowisk języka Python (tab.~\ref{tab:srodowiska}). Środowisko
aplikacji jest lekkie i nie zawiera biblioteki PyTorch; środowisko modeli służy do
segmentacji znamienia modelem M1, uczenia modeli oraz przetwarzania wsadowego. Ścieżki
środowiska modeli, wag i zbiorów danych zdefiniowano w module
\texttt{modele\_segmentacja/konfiguracja.py}; katalog bazowy można zmienić zmienną
środowiskową \texttt{MODELE\_DANE}.

\begin{table}[!ht]
\centering
\caption{Środowiska uruchomieniowe systemu}
\label{tab:srodowiska}
\footnotesize
\begin{tabularx}{\textwidth}{>{\raggedright\arraybackslash}p{2.2cm}
>{\raggedright\arraybackslash}X >{\raggedright\arraybackslash}X}
\toprule
& Środowisko aplikacji & Środowisko modeli \\
\midrule
położenie & \texttt{inzynierka\textbackslash} (folder główny projektu) &
\texttt{\%USERPROFILE\%\textbackslash modele\_dane\textbackslash srodowisko\textbackslash}
\\
Python & 3.14 & 3.12 \\
biblioteki & customtkinter, OpenCV, NumPy, Pillow, flirpy, matplotlib; SimpleITK (tylko
badanie E4) & PyTorch 2.11 z CUDA 12.8, torchvision, OpenCV, NumPy, bitsandbytes,
SimpleITK, onnx (lista: \texttt{modele\_segmentacja/wymagania\_srodowiska.txt}) \\
zastosowanie & aplikacja, model M3 (ONNX), testy & model M1, uczenie modeli, potok wsadowy,
raport zbiorczy \\
\bottomrule
\end{tabularx}
\end{table}

Instalacja środowiska aplikacji (w folderze głównym projektu):
\begin{verbatim}
py -3.14 -m venv inzynierka
inzynierka\Scripts\pip install customtkinter opencv-python numpy pillow ^
flirpy matplotlib simpleitk
\end{verbatim}
```

```
Środowisko modeli tworzy się analogicznie z pliku \texttt{wymagania\_srodowiska.txt}
(\texttt{pip install -r}). Wagi modeli umieszcza się w katalogu
\texttt{modele\_dane\textbackslash wagi}: koder PanDerm i wytrenowany model M1 w
podkatalogu modelu, a model M3 jako pliki \texttt{unet\_termal\textbackslash fold1.onnx}--
\texttt{fold5.onnx} i \texttt{wszystkie.onnx}.

\section{Struktura kodu}

Etapy systemu zrealizowano w pakiecie \texttt{analiza}, wspólnym dla aplikacji i
przetwarzania wsadowego (tab.~\ref{tab:moduly}). Modele segmentacji znajdują się w pakiecie
\texttt{modele\_segmentacja}, a metoda rejestracji -- w module \texttt{dopasowanie3.py}.

\begin{table}[!ht]
\centering
\caption{Moduły pakietu \texttt{analiza} i główne moduły projektu}
\label{tab:moduly}
\footnotesize
\begin{tabularx}{\textwidth}{>{\raggedright\arraybackslash}p{4.3cm} X}
\toprule
Moduł & Odpowiedzialność \\
\midrule
\texttt{analiza/system.py} & klasa \texttt{Analiza} (etapy potoku), wyjątek
\texttt{BladEtapu}, funkcje \texttt{analizuj}, \texttt{analizuj\_z\_zapisanych} \\
\texttt{analiza/dane.py} & dane par ze zbioru, adnotacje z CVAT, surowa intensywność z
nagrań .SEQ \\
\texttt{analiza/geometria.py} & przenoszenie punktów i masek macierzą $T$, ułamek pokrycia,
skala mm/piksel \\
\texttt{analiza/miary.py} & Dice, IoU, HD95, ASSD, przesunięcie środków, stosunek pól, TRE
\\
\texttt{analiza/pomiary.py} & parametry znamienia, piksele znamienia na obrazie
termowizyjnym i ich zapis do CSV \\
\texttt{analiza/obrazy.py}, \texttt{raport.py} & obrazy raportu i raport HTML z
objaśnieniami \\
\texttt{analiza/baza.py} & klasa \texttt{Baza} (SQLite), schemat, migracja, import i
eksport \\
\texttt{dopasowanie3.py} & detekcja znacznika, dopasowanie wstępne, ECC, wybór modelu
przekształcenia \\
\texttt{modele\_segmentacja/} & interfejs \texttt{ModelSegmentacji}, model M1, kandydaci na
znamię, model M3 (\texttt{unet\_termal/}) \\
\texttt{aplikacja.py} & interfejs graficzny (klasy \texttt{Aplikacja}, \texttt{Podglad}) \\
\bottomrule
\end{tabularx}
\end{table}

Klasa \texttt{Analiza} udostępnia metody odpowiadające etapom potoku, które należy
wywoływać w kolejności przedstawionej na listingu~\ref{lst:analiza}. Wywołanie etapu bez
wyniku poprzedniego kończy się wyjątkiem \texttt{BladEtapu} z opisem brakującego kroku.

\begin{lstlisting}[language=Python, caption={Analiza pary bez interfejsu graficznego},
label={lst:analiza}]
from analiza.system import Analiza
```

```
a = Analiza.z_pary(14) # albo Analiza("zdjecie.jpg", "termal.jpg")
a.rejestruj() # macierz T
a.ustaw_narozniki() # narozniki i skala mm
a.segmentuj_znamie() # model M1 (srodowisko modeli)
a.segmentuj_znacznik_termal() # model M3 (ONNX)
a.przenies(); a.zmierz() # przeniesienie ROI, pomiary, piksele
print(a.zapisz_raport()) # raport, metryki.json, CSV, wpis w bazie
\end{lstlisting}

\section{Formaty danych}

Dane pary przechowywane są w folderze \texttt{dane/pary/N}: zdjęcie \texttt{rgb.jpg}, obraz termowizyjny \texttt{termal.jpg}, surowe wartości detektora \texttt{termal\_surowe.npy} (macierz 240~$\times$~320, \texttt{uint16}), mapa temperatur \texttt{termal\_C.npy}, maski z CVAT (PNG, 0/255) oraz \texttt{narozniki.json} z narożnikami i homografiami piksele--milimetry obu modalności. Wynik rejestracji zapisywany jest w folderze \texttt{wyniki/wyniki\_dopasowania/para\_N}, m.in. jako macierz $T$ w pliku tekstowym \texttt{dopasowanie\_macierz.txt} (3~$\times$~3, piksel zdjęcia $\to$ piksel obrazu termowizyjnego). Plik \texttt{metryki.json} zawiera wyniki pogrupowane według badań (\texttt{rejestracja}, \texttt{zgodnosc}, \texttt{zgodnosc\_modalnosci}, \texttt{pomiary\_rgb}, \texttt{kontrola\_skali}, \texttt{piksele\_termal} i in.), a format pliku pikseli opisano w tabeli~\ref{tab:instrukcja-pliki}.

\section{Baza danych}

Schemat bazy (rys.~\ref{fig:erd}) zdefiniowano w module \texttt{analiza/baza.py}. Numer wersji schematu jest zapisywany w polu \texttt{user\_version} pliku bazy (obecnie 2); przy otwarciu starszej bazy tworzone są brakujące tabele. Obsługę bazy z wiersza poleceń zapewnia skrypt \texttt{baza\_danych.bat} z poleceniami \texttt{utworz}, \texttt{importuj\_wyniki}, \texttt{pokaz}, \texttt{eksport}, \texttt{eksport\_piksele} i \texttt{sql}. Przykładowe zapytania przedstawiono na listingu~\ref{lst:sql}.

\begin{lstlisting}[language=SQL, caption={Przykładowe zapytania do bazy danych}, label={lst:sql}]
-- wspolczynnik Dice'a otworu (badanie E2) dla wszystkich par
SELECT para, wartosc FROM wyniki_par
WHERE grupa = 'zgodnosc_modalnosci.otwor' AND nazwa = 'dice';
-- piksele znamienia pary 14 z ostatniej analizy
SELECT x, y, intensywnosc FROM piksel
WHERE w_masce = 1 AND analiza_id =
(SELECT id FROM ostatnia_analiza WHERE para_id = 14);
\end{lstlisting}

\section{Skrypty pomocnicze, uczenie i testy}

Skrypty wsadowe uruchamia się z folderu \texttt{projekt} (tab.~\ref{tab:skrypty}). Testy funkcjonalne (podrozdział~\ref{sec:testy-funkcjonalne}) uruchamia skrypt \texttt{testy/uruchom\_testy.bat}, który zapisuje wynik w pliku \texttt{testy/wynik\_testow.txt}.

\begin{table}[!ht]
\centering
\caption{Skrypty pomocnicze systemu}
\label{tab:skrypty}
```

```
\footnotesize
\begin{tabularx}{\textwidth}{>{\raggedright\arraybackslash}p{5.0cm} X}
\toprule
Skrypt & Zadanie \\
\midrule
\texttt{potok\_pary.bat 1-94} [\texttt{--z\_zapisanych}] & analiza wskazanych par (z
zapisaną rejestracją i maskami M1 -- bez GPU) \\
\texttt{raport\_zbiorczy.bat} & raport zbiorczy wszystkich par i tabele \LaTeX{} \\
\texttt{porownaj\_rejestracje.py} & badanie E4: porównanie metod rejestracji \\
\texttt{read\_flirpy.py}, \texttt{heic\_do\_jpg.py} & konwersja nagrań .SEQ i zdjęć HEIC \\
\texttt{temperatury\_par.py} [\texttt{--tylko\_surowe}] & mapy temperatur i surowe wartości
detektora dla par \\
\texttt{import\_cvat.py} & import adnotacji z programu CVAT \\
\texttt{modele\_segmentacja/trenuj\_panderm.bat} & douczanie modelu M1 na zbiorze ISIC 2018
\\
\texttt{modele\_segmentacja/trenuj\_unet\_termal.bat}, \texttt{ocen\_unet\_termal.bat} &
uczenie modelu M3 (5 foldów i model końcowy) i jego ocena \\
\bottomrule
\end{tabularx}
\end{table}

\section{Rozszerzanie systemu}

Nowy model segmentacji znamienia dodaje się jako podkatalog
\texttt{modele\_segmentacja/modele} z klasą \texttt{Model} dziedziczącą po
\texttt{ModelSegmentacji}; model jest wykrywany automatycznie. Nowa miara dopisana do
słownika wyników trafia automatycznie do pliku \texttt{metryki.json} i tabeli
\texttt{miara}, bez zmiany schematu bazy. Parametry metody rejestracji -- model
przekształcenia (\texttt{MODEL}), próg zgodności z wzorcem (\texttt{MIN\_ZGODNOSC}) i
rozdzielczość roboczą zdjęcia (\texttt{RGB\_DET\_PX}) -- zdefiniowano jako stałe w module
\texttt{dopasowanie3.py}.
```